RicozChange

AI-NATIVE CHANGE MANAGEMENT

v0.3 “Operations” — Product PRD

Four weeks to make RicozChange infrastructure: deploys that become changes by themselves, freezes on every calendar, approvals in Microsoft Teams, and the metrics a head of engineering screenshots for their boss.

VERSION	TIMELINE	DATE	BASELINE
v0.3 (Operations)	4 weeks · 1 dev + AI agents	September 27, 2026	v0.2 complete — auth, Slack, email, hardening

Confidential — internal team document

01

Why v0.3 exists

v0.2 made RicozChange something a real team can run its week on. v0.3 makes it **infrastructure**: the product stops being a place people *fill in* and becomes a system that *watches the deployment pipeline*, talks to the calendars people already live in, meets Windows-heavy enterprises on Microsoft Teams, and produces the metrics a head of engineering screenshots for their boss.

Four features, one per week

1. **Deploy-as-change**
   A production deploy auto-creates the change request. Zero data entry.
2. **Calendar sync**
   Freezes and change windows live on the calendars people already check.
3. **Microsoft Teams**
   Same approval cards as Slack, for the enterprise half of the market.
4. **DORA-style analytics**
   CFR, approval latency, MTTR by team/system — exportable.

Goals

- A push to `main` on a connected repo becomes a scored, collision-checked change with no human filling any form — and the deploy status records the outcome.
- Freeze windows subscribe-able as an .ics feed; two-way write to Google Calendar / Outlook.
- Teams approval cards with the same any-of policy, identity mapping and audit trail as Slack.
- A metrics dashboard with CSV export, computed entirely from existing data.

Non-goals (explicitly deferred)

Multi-workspace Slack, Jira/ServiceNow sync, learned ML risk weights, mobile push, billing/seats and the compliance pack (v1.0), on-prem installs.

Where v0.2 left us

Already shipped	What it gives v0.3
Clerk sign-in + server-enforced roles	Real identity on every new surface
Slack approvals (OAuth, signed callbacks, idempotent buttons)	The exact pattern Teams ports in week 3
Email-to-change + automated sweeps	Parser, dedupe and fallback-chain patterns to reuse
Rate limiting, logging, background sweep, smoke test	Hardening already in place for three new webhooks
67 passing tests + `scripts/smoke.py`	Regression safety while internals get rewired

02 · WEEK 1

Deploy-as-change — the flagship

A push to production on a connected repo **becomes** a change request: scored, collision-checked, approved if risky, and its outcome recorded when the deploy finishes. Engineers never fill the form.

User stories

- As an engineer, when I deploy to production from GitHub Actions, a change request appears with the commit/PR linked — I never fill the form.
- As an approver, the auto-created change goes through the same scoring, collision detection and (if risky) the same Slack/email approval flow as a hand-filed change.
- As an owner, when the deploy job finishes, the outcome (success/failed) is recorded from the deploy status — CFR feeds itself.

Technical design

Piece	Design
Webhook	`POST /api/integrations/github/webhook` — HMAC `X-Hub-Signature-256` verification, reuses the existing rate limiter; `workflow_run` / `deployment` events only.
Repo connections	New `github_connections` table (repo, default systems, auto-submit threshold, secret) + admin UI on the Integrations page.
Mapping rules	Deploy env `production` → change type from changed paths / commit scope (fallback `normal`); deploy ref + run URL → description; deployer email → owner via existing email matching.
Outcome loop	Deploy completes → `record_post_change(success|failed)` from the `workflow_run` conclusion — the "did it work?" loop closes itself for deploys.
Idempotency	Unique `delivery_id` per event — same pattern as the email `Message-ID` dedupe.
Duplicate safety	A deploy matching an open human-filed change on the same systems links to it instead of creating a duplicate.

Acceptance criteria

1. Push to a connected repo → change created within 10 s, scored, systems attached, owner mapped.
2. Risky auto-change → approvers get the same Slack DM with the "why".
3. Deploy failure → `post_change_result=failed`; the CFR dashboard moves.
4. Replayed webhook delivery → no duplicate change.
5. Invalid signature → 401 + audit entry.

03 · WEEKS 2–3

Calendar sync & Microsoft Teams

Calendar sync (week 2)

Freezes and change windows appear on the calendars people already check.

- **Read-only out (day 1):** `GET /api/calendar/changes.ics` — RFC 5545 feed of change windows + freezes, token-authenticated per user (`users.calendar_token`, refreshable). No external dependencies — this alone delivers most of the value.
- **Two-way write (rest of week):** Google Calendar API + Microsoft Graph (OAuth; tokens in the `settings` table — same pattern as the Slack bot token). Changes create events; freezes create all-day events. Calendar errors never block the workflow (logged, retried by the existing sweep pattern).
- **Propagation:** editing a change's window updates the linked event (`calendar_event_id` stored on the change).

Acceptance criterion	Demo gate
.ics URL subscribes in Google Calendar and Outlook; events show window + risk	Change appears in a real calendar within 60 s; old token dies on rotation; calendar outage breaks nothing
Rotating the token kills the old URL (audit-logged)	
A submitted change appears in the connected calendar within 60 s	
Calendar outage → nothing in the change workflow breaks	

Microsoft Teams (week 3)

The Slack approval experience, ported — same policy, same audit, new transport.

- Azure Bot registration + Bot Framework; `POST /api/teams/interactions` validates the bot token against Azure's published OpenID metadata (cached) — the Slack-signature pattern ported.
- Outbound proactive messages reuse the existing outbox shape (`Notification.kind="teams"`) so the Approvals page renders Teams rows without new frontend logic.
- Identity: `users.teams_id`; same email auto-link + ephemeral fallback as Slack.
- Any-of policy, idempotency keys and delivery audit are literally the same service calls. Teams outage → the email fallback chain already built in v0.2.

Acceptance criterion	Demo gate
Approve from a phone in Teams in < 30 s; duplicate clicks never double-apply	Teams approval on a phone in < 30 s; invalid token → 401 + audit; outage → email fallback
Both approvers' cards close correctly under any-of (one approves, other superseded)	
Invalid bot token → 401 + audit entry; Teams outage → email fallback fires	

04 · WEEK 4

DORA-style analytics & the plan

Analytics (week 4)

The dashboard a head of engineering screenshots for their boss.

Metric	Definition (fixed here, printed on the page)
Change failure rate	Failed + partial post-change results ÷ all recorded results (exists — extended with grouping)
Lead time to approve	Median: change created → approved
Approval latency	Median: submitted → decided
MTTR	For failed changes: window end → the follow-up change recording success
Volume & risk mix	Changes per week by class; share of standard/normal/major/emergency

- `GET /api/analytics?from&to&group_by=system|team` — all computed from existing tables; schema change is indexes only.
- Frontend: an **Analytics** page under Operate using the existing chart components; date-range picker; `GET /api/analytics/export.csv` reusing the importer's CSV conventions (Excel-clean).
- Aggregate-only for non-admins (per-team visibility rules arrive with v1.0 workspaces).

Week-by-week plan

Week	Deliverable	Demo gate
1	GitHub webhook, repo connections, auto-scoring, deploy-outcome loop	Push to a sample repo → scored change → Slack approval → deploy status records the outcome, no form touched
2	.ics feeds (day 1), Google/Outlook two-way sync, token rotation	Change appears in a real calendar within 60 s; old token dies on rotation
3	Teams app, approval cards, identity mapping, fallback chain	Approve from Teams on a phone in < 30 s; duplicates idempotent
4	Analytics API + page + CSV export, hardening, E2E smoke extension, docs	Manager demo: push → auto-change → Teams approval → CFR moves on the analytics page

Effort: W1 ~7 dev-days · W2 ~5 · W3 ~6 · W4 ~4 (solo + AI agents, tests included).

05

Risks, success metrics & what's deferred

Top risks and mitigations

Risk	Mitigation
GitHub event variety (rename, re-run, skipped runs)	Only two event types; unique `delivery_id`; unknown payloads rejected with 4xx, never 500
Calendar APIs eat the week	Ship the .ics feed on day 1 — value exists even if OAuth write slips; two-way is a bonus
Azure bot registration friction (tenant, consent)	Start registration in week 1 in parallel; demo on the Bot Framework Emulator until tenant consent lands
Metric definitions argued about	Definitions fixed in this PRD and printed on the Analytics page itself
Webhook attack surface grows	Every new webhook: signature verification + the existing rate limiter + idempotency — same as email

Success metrics (30 days after rollout)

- ≥ 30% of changes auto-created from deploys; time-to-file < 30 s for deploy changes
- ≥ 1 external calendar subscriber per team; zero workflow incidents caused by calendar outages
- ≥ 20% of approvals decided from Teams in Teams-first shops
- The analytics page becomes the most-screenshotted page in manager demos

Deliberately deferred

Billing/seats, compliance pack, SSO hardening, workspaces with per-team visibility, ML risk weights, Jira/ServiceNow sync, mobile push — v1.0 candidates; none block the operations story.

The version ladder, updated

Version	Theme	State
v0.2	"Real team"	**Shipped:** sign-in & roles, Slack, email, hardening, smoke test
v0.3	"Operations"	**This PRD:** deploy-as-change, calendar, Teams, analytics — 4 weeks
v1.0	"Pilot — sellable"	Compliance pack, billing, SSO hardening, onboarding wizard (~6 weeks after v0.3)